

A Cognitive-Emulation Architecture for Validated Decision-Making in Digital Twins: Application to Human-Robot Collaboration Safety

Ibrahim Yousif¹, Ahmed Mahmoud², Michael Wise¹,
Leo Lyublinski³, Amanda White⁴, Arthur Carvalho⁴,
Lora A. Cavuoto⁵, Mohammad Mayyas¹, Fadel M. Megahed^{4*}

¹Department of Engineering Technology, Miami University, 101
Knightsbridge Dr., Hamilton, 45011, OH, USA.

²Department of Mechanical Engineering, University of South Carolina,
Columbia, SC, 29201, USA.

³College of Arts and Science , Miami University, 100 Bishop Circle,
Oxford, 45050, OH, USA.

⁴Farmer School of Business, Miami University, 800 E. High St., Oxford,
45050, OH, USA.

⁵Department of Industrial and Systems Engineering, University at
Buffalo, 407 Bell Hall, Buffalo, NY, 14260, USA.

*Corresponding author(s). E-mail(s): fmegahed@miamioh.edu;

Abstract

Digital twins promise closed-loop decision-making in manufacturing, yet many implementations remain digital shadows that monitor without validating candidate interventions before execution. This paper presents a cognitive-emulation digital twin architecture that addresses this gap. The architecture decouples perception and inference (a cognitive twin) from constraint-based validation and

execution (an emulation twin), so candidate decisions are screened for feasibility and risk before reaching the physical system. The pattern is domain-agnostic and applies wherever decisions must be validated under operational constraints such as timing, kinematics, collision, or resource availability. We instantiate and evaluate the architecture in human–robot collaboration (HRC) safety, a demanding setting where interventions must be safe and physically feasible under real-time latency. The cognitive twin converts overhead vision streams into workspace state and risk cues, while the emulation twin validates speed, stop, and workspace-constraint recommendations against kinematic reachability and predicted collisions. A laboratory prototype in a collaborative assembly workcell with a UR5e robot demonstrates end-to-end operation from perception to validated intervention. The work clarifies the architectural role of emulation in advancing digital twins from shadows to actionable systems and provides practical deployment considerations for manufacturing practitioners.

Keywords: cognitive manufacturing, cognitive-emulation architecture, digital twins, human-robot collaboration, occupational safety, validated decision-making

1 Introduction

Smart manufacturing increasingly depends on real-time decision-making, whether for safety intervention, production scheduling, predictive maintenance, or resource allocation. Digital twins promise to support such decisions by coupling sensing, models, and simulation into a bidirectional link between the physical system and its virtual counterpart (Grieves & Vickers, 2017; Tao et al., 2018). Yet systematic reviews have consistently reported that many deployed digital twins function primarily as digital shadows. They monitor, visualize, or support offline what-if analysis, while decision gating and bidirectional actuation remain less common and often under-specified (Gálvez del Postigo Gallego et al., 2025; Iranshahi et al., 2025; Macháček et al., 2025; Pronost et al., 2024; Seipolt et al., 2025). Across manufacturing, most reported implementations remain prototypes oriented toward observability and analysis rather than validated intervention (Gálvez del Postigo Gallego et al., 2025; Iranshahi et al., 2025; Pronost et al., 2024). This paper addresses that gap.

The term *digital twin* is also used inconsistently across manufacturing applications. For example, [Grieves and Vickers \(2017\)](#) described a coupled physical and virtual representation that is updated over time and used to support decisions across the lifecycle. Subsequent characterizations identified core properties essential to digital twin operation. [Jones et al. \(2020\)](#) noted that the virtual-to-physical connection (i.e., the mechanism through which decisions reach the physical system) remains underdeveloped in practice. Standardization bodies have since established manufacturing reference architectures (e.g., ISO 23247), yet published implementations still vary in scope and in what they consider a “twin” ([International Organization for Standardization, 2021](#); [Iranshahi et al., 2025](#); [Macháček et al., 2025](#); [Shao, 2024](#)). Deployment architecture further shapes what a twin can do in practice ([Z. Li et al., 2025](#)).

A common taxonomy helps clarify what a twin can and cannot do. [Kritzinger et al. \(2018\)](#) distinguished a *digital model* (no automated linkage), a *digital shadow* (automated one-way updates), and a *digital twin* (automated bidirectional linkage that enables the virtual system to influence the physical system). This distinction matters for decision-making. A digital shadow can support awareness, but it does not, by itself, enable validated intervention. Recent industrial applications have reinforced the role of emulation and virtual commissioning as practical mechanisms for validating control decisions before deployment ([Langås et al., 2026](#); [Macháček et al., 2025](#)).

Even accurate perception and sound inference can produce infeasible or unsafe outcomes when candidate actions are not checked against operational constraints before reaching the physical system. The problem is general. A scheduling twin must account for equipment health and failure uncertainty when sequencing jobs ([Negri et al., 2021](#)). A quality-control twin must confirm that an adjustment stays within process limits. A safety twin must ensure that a stop or speed-change command is reachable and collision-free under latency. [Lugaresi et al. \(2023\)](#) formalized online validation for manufacturing digital twins and highlighted that validity must be assessed rapidly, with

limited data, in dynamic contexts. Across domains, the core requirement is the same: candidate decisions need a validation gate (emulation twin) that checks feasibility and constraint satisfaction before execution.

This paper introduces a *cognitive-emulation* digital twin architecture that addresses this requirement. The architecture separates two roles. A **cognitive twin** ingests sensor data, estimates state, and generates candidate decisions. An **emulation twin** then acts as a *validation gate* by checking each candidate against operational constraints (kinematic, collision, timing, and resource) before any action reaches the physical system. The separation is deliberate. The cognitive twin can evolve independently (e.g., swapping a vision model for a different sensing modality) without changing the validation logic. The emulation twin can enforce domain-specific constraints without coupling to a particular perception stack. The architecture is compatible with common industrial simulation and digital twinning workflows. For example, Process Simulate X supports simulation-driven feasibility studies and safety-zone design (Siemens, 2026a, 2026b), while Omniverse and Isaac Sim document workflows for assembling and simulating robotics-focused digital twins (NVIDIA, 2026a, 2026b).

We use human-robot collaboration (HRC) safety to stress-test our architecture under timing and physical-feasibility constraints. In HRC, decisions must respond quickly, protect workers, and respect robot capabilities such as reachability and collision-free motion (International Organization for Standardization, 2016, 2021; Marvel & Norcross, 2017; Szabo et al., 2012). Surveys of safe HRC have emphasized that the system must sense the human, anticipate motion, and adapt robot behavior at runtime (W. Li et al., 2024). Recent reviews of collaborative robots in manufacturing have reinforced the centrality of safety across assembly and disassembly applications (Keshvarparast et al., 2024). HRC safety, therefore, provides a rigorous test of the cognitive-emulation architecture because every component of the pipeline,

from perception through validation to execution, must operate under tight timing and feasibility constraints.

Computer vision is a practical sensing modality for data-driven safety monitoring since it can monitor wide areas, infer spatial relationships, and detect unsafe proximity or non-compliance events. Prior work has demonstrated vision-enabled hazard assessment and monitoring logic for collaborative tasks (Alenjareghi et al., 2025; Chebotareva et al., 2025; Faqueer & Khajavi, 2025; Saleem et al., 2025). These studies showed what sensing can deliver, but they also highlighted what is often missing: a mechanism to select and validate interventions before execution. Recent work on forecasting-enabled collision detection has further illustrated how models can support proactive decisions (El Kalach et al., 2026). However, accurate perception can still lead to unsafe outcomes if candidate actions ignore latency, uncertain human motion, or robot kinematic constraints (Marvel & Norcross, 2017). Broader trust and security considerations for digital twin deployments compound this risk (Voas et al., 2025).

In our HRC demonstration, the cognitive twin converts overhead vision streams into workspace state (human pose, robot state, object locations) and risk cues, which feed a candidate response generator that proposes interventions such as speed reduction, safety-rated stop, workspace-constraint updates, or replanning requests. The emulation twin then validates each candidate against kinematic reachability, predicted collisions under human motion uncertainty, and latency constraints before any action reaches the physical workcell. Fig. 1 illustrates this closed-loop pipeline. While demonstrated here in HRC safety, the architecture is domain-agnostic. Other applications would substitute domain-appropriate perception, candidates, and constraints.

We implement a preliminary prototype of the framework in a laboratory collaborative assembly workcell with a UR5e robot and overhead vision sensing. This implementation example illustrates how cognitive inference and emulation-based validation can work together. We do not claim field deployment, and we do not claim

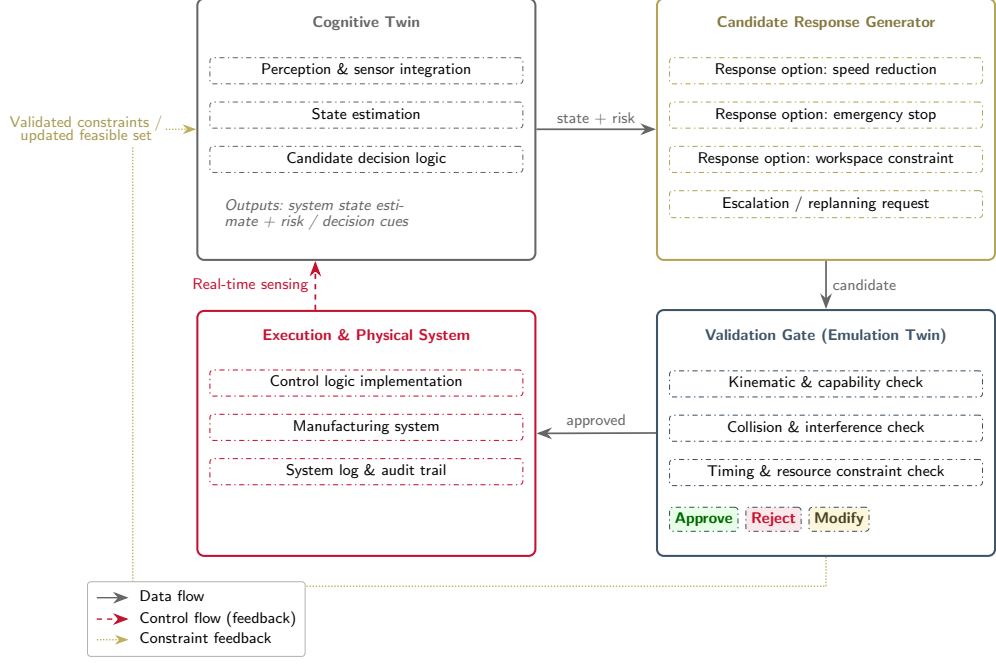


Fig. 1 Cognitive-emulation digital twin architecture for validated decision-making, demonstrated here in human–robot collaboration. The **cognitive twin** (top left) ingests sensor data and produces state estimates and risk cues, which feed the **candidate response generator** (top right). Candidates flow down to the **validation gate** (bottom right), which checks each against operational constraints (kinematic reachability, collision, latency, and related timing/resource constraints) before approved actions reach the **execution layer** (bottom left). Dashed red arrows denote real-time sensing feedback; dotted gold arrows denote constraint feedback from the validation gate to the cognitive twin.

fully real-time motion planning. Our goal is to simply showcase how rapid validation checks can inform timely decisions in safety-critical manufacturing tasks.

Taken together, our paper makes the following contributions:

1. Recent reviews have shown that most digital twins remain shadows, lacking validated decision-making (Gálvez del Postigo Gallego et al., 2025; Kritzinger et al., 2018; Pronost et al., 2024; Seipolt et al., 2025). We present a cognitive-emulation architecture that decouples perception and inference from constraint-based validation and execution. We instantiate the architecture in HRC safety to clarify how emulation advances digital twins from shadows to actionable systems.
2. Existing vision-based safety studies have focused on hazard detection but stopped short of closed-loop intervention (Alenjareghi et al., 2025; Chebotareva et al., 2025).

We develop a vision-driven cognitive twin that produces real-time workspace state and risk cues for safety decision logic in collaborative workcells.

3. Prior work identified the need for rapid online validation in manufacturing twins (Lugaresi et al., 2023) and documented the role of latency and stopping-distance constraints in HRC (Marvel & Norcross, 2017). We introduce an emulation-twin validation layer that checks feasibility and collision constraints for candidate responses under these considerations.
4. Standards and trust frameworks have highlighted the gap between prototype demonstrations and production-ready deployments (Shao, 2024; Voas et al., 2025). We provide a laboratory implementation in a collaborative assembly cell and document practical deployment considerations for manufacturing practitioners.

The remainder of the paper is organized as follows. Section 2 reviews related work on digital twin architectures and standards, decision support in manufacturing twins, vision-based safety monitoring, and HRC safety. Section 3 presents the cognitive-emulation architecture in domain-neutral terms and then details the HRC-specific cognitive and emulation components and their interfaces. Section 4 describes the experimental workcell, datasets, and evaluation methodology, followed by results. Section 5 provides a discussion of implications, limitations, and deployment considerations. Section 6 presents our concluding remarks.

2 Related Work

2.1 Digital Twin Maturity: From Shadows to Actionable Twins

Digital twin adoption in manufacturing has grown substantially, driven by the promise of coupling physical systems with virtual counterparts that can support monitoring, analysis, and decision-making across the product lifecycle (Tao et al., 2018). Jones et al. (2020) characterized the digital twin through thirteen core properties,

including physical and virtual entities, bidirectional twinning, and metrology, and found that the virtual-to-physical connection remains the least developed in practice. Despite broad adoption, implementations vary widely in what they actually deliver, and the gap between conceptual frameworks and deployed capabilities persists across manufacturing domains (Iranshahi et al., 2025; Macháček et al., 2025).

Kritzinger et al. (2018) provided a taxonomy that clarifies this variation. A *digital model* has no automated data exchange between physical and virtual representations. A *digital shadow* automates one-way updates from the physical system to the virtual one. A *digital twin* adds automated bidirectional exchange so that the virtual representation can influence the physical system. Of the 42 publications they reviewed, only 18% described true digital twins with bidirectional data flow, while 35% were digital shadows and 28% were digital models. Recent surveys reinforce this finding. Pronost et al. (2024) reported that 58% of reviewed manufacturing implementations are digital shadows, with only 18% qualifying as full twins. Seipolt et al. (2025) taxonomized 242 articles and found that most implementations remain at the descriptive or diagnostic level rather than reaching prescriptive or autonomous operation.

Architectural proposals have attempted to bridge this gap. Redelinghuys et al. (2020) organized the digital twin into six functional layers, from physical sensing through data communication to a top-level cognition layer responsible for emulation, simulation, and decision support. Their architecture demonstrates that the transition from shadow to twin depends on populating the upper layers: specifically, the cognition layer (where inference and analysis occur) and the actuation pathway (where validated decisions return to the physical system). Platform selection also shapes what a twin can do in practice; Dodero et al. (2025) evaluated sixteen development environments across robotics, game, and simulation engines, and found that no single platform covers all requirements. Their findings reinforce the need for modular architectural thinking.

These findings indicate that most deployed digital twins remain shadows because two capabilities are often missing in practice: (i) a cognitive layer that infers actionable state from raw observations, and (ii) an online validation layer that screens candidate decisions for feasibility before execution. Table 1 summarizes the related work positioned against these architectural roles using each study’s stated scope and definitions. Columns 1–4 correspond to the four roles formalized in Section 3. The sparsity of checkmarks in the candidate-generation and constraint-validation columns motivates the architecture presented in this paper.

Table 1 Summary of related work positioned against the cognitive-emulation architecture. Columns correspond to the four architectural roles formalized in Section 3. ✓ = explicitly addressed; (✓) = partially addressed; — = not addressed.

Reference	<i>State estimation</i>	<i>Candidate generation</i>	<i>Constraint validation</i>	<i>Execution feedback</i>	<i>Real-time</i>	Domain
Kritzinger et al. (2018)	—	—	—	—	—	Taxonomy
Tao et al. (2018)	✓	—	—	—	—	Manufacturing
Jones et al. (2020)	(✓)	—	—	—	—	Survey
Redelinghuys et al. (2020)	✓	(✓)	—	✓	(✓)	Manufacturing
Tong et al. (2020)	✓	—	✓	(✓)	✓	Machining
Negri et al. (2021)	✓	✓	(✓)	✓	✓	Scheduling
Lugaresi et al. (2023)	✓	—	(✓)	—	✓	Manufacturing
Wang et al. (2023)	✓	(✓)	—	(✓)	✓	HRC
Cimino et al. (2024)	✓	(✓)	(✓)	—	—	HRC
Chand et al. (2024)	✓	—	—	—	✓	HRC
Pronost et al. (2024)	(✓)	—	—	—	—	Survey
Gálvez del Postigo Gallego et al. (2025)	(✓)	(✓)	—	—	—	Survey
Ramkumar and Balasubramanian (2025)	✓	(✓)	—	—	✓	HRC
Alenjareghi et al. (2025)	✓	(✓)	—	—	✓	HRC
El Kalach et al. (2026)	✓	(✓)	(✓)	—	✓	HRC
This paper	✓	✓	✓	✓	✓ (online, lab)	HRC (demonstration)

2.2 Online Validation and Virtual Commissioning

Digital twins can validate decisions at two stages. Virtual commissioning verifies system designs and control logic offline, before deployment, using simulation models that approximate the target environment (Macháček et al., 2025). On the other hand, online validation screens decisions during operation, when the physical system is running, and conditions may deviate from design assumptions. Cimino et al. (2024) illustrated the offline case: their simulation-based digital twin evaluated alternative task allocations for HRC assembly through discrete-event simulation, enabling what-if analysis of working configurations before implementation. This approach shows what simulation can deliver at the planning stage, but does not screen individual actions at runtime.

Lugaresi et al. (2023) formalized online validation for manufacturing digital twins. Their method compared execution traces from the physical system against predictions from the simulation model, using sequence-comparison techniques (longest common subsequence and dynamic time warping) to detect divergence. Validation is rapid and continuous, operating with limited data while the system runs. The critical distinction from our work is the object of validation. Lugaresi et al. (2023) validated the twin itself (is the model still accurate?). On the other hand, our architecture validates the action (is this candidate feasible given the current state?). Model fidelity and action feasibility are complementary concerns, but they require different mechanisms.

Two studies illustrated domain-specific validation approaches that inform our formulation. Negri et al. (2021) synchronized a scheduling digital twin with real-time equipment-health data and used a Monte Carlo simulation to evaluate schedule robustness under stochastic failures. Their probabilistic evaluation of candidate schedules parallels the chance-constrained formulation we adopt in Section 3.3. Tong et al. (2020) built a real-time machining digital twin that monitored axis motion ranges, spindle loads, and cutting forces against machine constraints during operation. Their

constraint-checking logic exemplifies the kind of constraint function $c_i(a, \hat{x}_t) \leq 0$ that our validation gate generalizes.

These approaches each address a piece of the validation problem: model fidelity, schedule robustness, or machine-level constraint monitoring. However, they do not provide a general-purpose gate that (i) receives an explicit set of candidate actions and (ii) screens each candidate against multiple constraint types before execution. This gap motivates our architectural separation between candidate generation in the cognitive twin (Section 3.2) and constraint-based screening in the emulation twin (Section 3.3).

2.3 Cognitive Digital Twins and Decision Support

A cognitive digital twin augments the standard physical–virtual pairing with AI or ML capabilities for perception, reasoning, and decision support. The concept traces to [Grieves and Vickers \(2017\)](#), who envisioned a digital twin that can simulate system responses to candidate scenarios and thereby mitigate unpredictable emergent behavior in complex systems. [Gálvez del Postigo Gallego et al. \(2025\)](#) surveyed how decision support is realized within manufacturing digital twin ecosystems and found that most implementations provide descriptive or diagnostic analytics; prescriptive decision-making, where the twin recommends and validates specific actions, remains uncommon. This aligns with the shadow-dominated landscape described in Section 2.1.

Several recent systems implement cognitive capabilities that map to specific blocks of our architecture. [Ramkumar and Balasubramanian \(2025\)](#) trained a deep CNN to estimate trust levels in HRC by detecting robot and human positions from synthetic and real images. Their system performs state estimation (the perception function $f(\cdot)$ in our notation) but does not generate candidate actions or validate them against operational constraints. [Chand et al. \(2024\)](#) built a vision-enabled human digital twin that monitored operator fatigue through action segmentation and physiological signals and feeds fatigue estimates into a genetic-algorithm scheduler. This covers state

estimation and offline task scheduling, but does not validate individual robot actions against kinematic or collision constraints at runtime. [Z. Li et al. \(2025\)](#) proposed a cloud–fog–edge computing framework that organizes digital twin services across network layers, addressing deployment architecture but not the content of validation logic. [Pronost et al. \(2024\)](#) scoped where cognitive capabilities appear across the product lifecycle and confirm that decision-support functions cluster in the production phase, yet their review does not identify any system that screens candidate actions through a dedicated feasibility gate.

Across studies, cognitive digital twins generate situational awareness, trust estimates, or recommendations, but prescriptive decision pipelines are typically not paired with an explicit feasibility screen that sits between recommendation and actuation. Our architecture addresses this by inserting a constraint-based validation gate between cognitive outputs and the execution layer (Section 3.3).

2.4 Safety in Human–Robot Collaboration

Collaborative robot safety is governed by ISO 10218 ([International Organization for Standardization, 2016](#)) and ISO/TS 15066, which define four collaborative modes including speed-and-separation monitoring. [Marvel and Norcross \(2017\)](#) implemented speed-and-separation monitoring on a testbed and documented the practical constraints that shape real-time safety decisions: sensor latency, stopping distance, and the trade-off between conservative safety margins and productive throughput. [Szabo et al. \(2012\)](#) described a complementary testbed for evaluating speed-and-separation monitoring implementations and highlighted the need for repeatable benchmarks. These studies establish the engineering context that safety interventions must be physically executable within tight timing budgets, and any proposed action (speed reduction, stop, or workspace constraint) must respect robot kinematics and braking dynamics.

Vision-based approaches extend the sensing side of this pipeline. [Alenjareghi et al. \(2025\)](#) integrated job hazard analysis with real-time computer vision (YOLO v5) to detect and classify hazards in a collaborative disassembly cell. [Chebotareva et al. \(2025\)](#) used monocular cameras with a three-level danger classifier (safe, attention, danger) to trigger automatic speed adjustment or emergency stop. [Saleem et al. \(2025\)](#) surveyed external sensor technologies for human detection in HRC and confirmed that camera-based systems can achieve high detection accuracy. These studies demonstrate what perception can deliver, but they share a common limitation: the path from detected hazard to executed intervention is either hard-coded (fixed speed reduction upon detection) or left unspecified. None evaluated whether the chosen intervention is kinematically reachable or collision-free under the current state.

Digital-twin-based safety systems add modeling and prediction to the pipeline. [El Kalach et al. \(2026\)](#) combined time-series forecasting (N-BEATS, GRU) with a digital twin to predict future robot joint trajectories and detect collisions before they occur. Their forecasting-enabled approach is the closest to proactive safety among the reviewed HRC systems, but it predicts collisions rather than generating and validating candidate interventions. [Wang et al. \(2023\)](#) proposed a digital-twin-based safety management framework for Industry 5.0 that uses ontology-based semantic reasoning to detect and classify unsafe states. The framework formalizes safety knowledge but does not structure constraint validation as a separate architectural layer. [Langås et al. \(2026\)](#) explored the synergy of human–robot teaming, digital twins, and machine learning, emphasizing virtual commissioning as a pathway to safer collaboration. [Keshvarparast et al. \(2024\)](#) surveyed collaborative robots across assembly and disassembly applications and confirmed that safety remains the central design concern.

The HRC safety literature thus progresses from standards-based monitoring through vision-enabled detection to digital-twin-assisted prediction. What remains

under-specified in most systems is an end-to-end decision architecture in which candidate interventions are generated and then screened through a dedicated feasibility gate that accounts for kinematic, collision/interference, and timing constraints before execution. Table 1 shows that no reviewed HRC system combines all four roles (state estimation, candidate generation, constraint validation, and execution feedback) within a unified pipeline. Our cognitive-emulation architecture fills this gap by explicitly separating decision proposal (cognitive twin) from constraint screening (emulation twin) prior to actuation.

Table 1 shows that state estimation is well represented as fourteen of the fifteen reviewed systems include it fully or partially. Candidate generation, by contrast, receives full treatment in only one system (Negri et al., 2021), with several others providing partial coverage through predefined response sets. Constraint validation as a dedicated architectural role is rarer still. Only Tong et al. (2020) addressed it directly through real-time machine-constraint monitoring, while Negri et al. (2021) and El Kalach et al. (2026) provided partial coverage through probabilistic schedule evaluation and collision prediction, respectively. No reviewed system combined all four roles (state estimation, candidate generation, constraint validation, and execution feedback) within a unified architecture. Our cognitive-emulation framework is designed to close this gap, as formalized in Section 3.

3 Cognitive–Emulation Architecture

This section presents the cognitive-emulation digital twin architecture in domain-neutral terms and links the notation directly to the information flow in Fig. 1. Section 3.1 defines the closed-loop decision setting and the core variables used throughout Section 3. We then detail the cognitive twin (Section 3.2), the validation gate implemented by the emulation twin (Section 3.3), the execution layer (Section 3.4),

and the interfaces and system considerations (Section 3.5). Section 3.6 instantiates the architecture for human–robot collaboration as a demonstration case.

3.1 Design Objectives and Architectural Overview

Fig. 1 depicts the architecture as a closed-loop decision system in which sensing updates a virtual representation, the system proposes candidate actions, and a validation gate screens candidates before execution. We formalize this loop using a compact set of variables that remain domain-agnostic.

At decision time t , the physical system is characterized by a state

$$x_t \in \mathcal{X}, \quad (1)$$

where x_t collects the operational variables relevant to decision-making (e.g., machine configuration, process variables, resource status, and environmental context). Sensors provide observations

$$y_t = H(x_t) + v_t, \quad (2)$$

where $H(\cdot)$ denotes the sensing process and v_t represents measurement noise and uncertainty. These observations correspond to the sensing feedback loop shown in Fig. 1.

The digital twin maintains an estimated state

$$\hat{x}_t = f(y_{0:t}), \quad (3)$$

where $f(\cdot)$ denotes perception, data fusion, and state inference applied to the observation history. Based on $\hat{x}_t$, the decision-making logic generates a finite set of candidate actions,

$$\mathcal{A}_t^{\text{cand}} = \pi(\hat{x}_t), \quad (4)$$

where $\mathcal{A}_t^{\text{cand}} = \{a_t^{(1)}, \dots, a_t^{(K)}\}$ contains discrete decision alternatives appropriate to the application. In Fig. 1, these candidates flow from the cognitive twin through the candidate response generator to the validation gate. The validation gate then selects an executable action a_t from the candidate set by enforcing operational constraints before forwarding an approved action to the execution layer.

After execution, the physical system evolves according to

$$x_{t+1} = F(x_t, a_t) + w_t, \quad (5)$$

where $F(\cdot)$ represents system dynamics and w_t captures disturbances, model mismatch, and external variability. New observations are collected and the loop repeats, which yields the closed-loop behavior in Fig. 1. This receding-horizon structure is standard in constrained model predictive control (Mayne et al., 2000). Our architecture adapts it to a digital twin setting where perception (Tao et al., 2018) and simulation-based validation (Lugaresi et al., 2023) replace the traditional optimizer.

3.2 Cognitive Twin: State Estimation and Candidate Generation

The cognitive twin implements the estimation function $f(\cdot)$ and the candidate generation policy $\pi(\cdot)$ defined in Eqs. (3)–(4). Its role is to convert raw observations into a structured state representation and a small set of plausible decision alternatives.

State estimation

The cognitive twin fuses sensor observations and system information to produce the operational state estimate $\hat{x}_t = f(y_{0:t})$ from Eq. (3). Depending on the application, $\hat{x}_t$ may include machine configuration, process variables, resource availability, environmental context, or predicted future conditions. The estimation process may combine

perception, data fusion, filtering, and learned models. In Fig. 1, this functionality corresponds to the perception and state estimation components of the cognitive twin.

Candidate generation

Based on $\hat{x}_t$, the cognitive twin generates the finite candidate set $\mathcal{A}_t^{\text{cand}} = \pi(\hat{x}_t)$ from Eq. (4). Each candidate represents a plausible operational response appropriate to the current situation. We intentionally generate a small discrete set rather than performing continuous optimization. This design enables the emulation twin to evaluate each alternative under tight latency constraints and supports predictable real-time behavior. In Fig. 1, these candidates flow from the cognitive twin through the candidate response generator to the validation gate.

Candidate response generator

The candidate response generator in Fig. 1 is a lightweight orchestration layer that packages candidates and handles regeneration requests between the cognitive twin and the validation gate. It is not part of the cognitive twin itself; rather, it receives the estimated state and decision cues from the cognitive twin and assembles them into the discrete candidate set $\mathcal{A}_t^{\text{cand}}$. If the validation gate rejects all candidates or returns updated constraints, the generator produces a revised set under the tightened feasible region.

3.3 Validation Gate (Emulation Twin): Constraint-Based

Decision Screening

Section 3.1 formalized the closed-loop decision setting and defined the estimated state $\hat{x}_t$ and the candidate set $\mathcal{A}_t^{\text{cand}}$ shown in Fig. 1. Section 3.2 described how the cognitive twin produces $\hat{x}_t$ and $\mathcal{A}_t^{\text{cand}}$. In this section, we define and implement the validation function $V(a, \hat{x}_t)$ used by the emulation twin to screen candidate actions against predictive constraint checks before any action reaches the execution layer.

At decision time t , each candidate action $a \in \mathcal{A}_t^{\text{cand}}$ is evaluated with respect to a set of operational constraints. We represent these requirements using constraint functions

$$c_i(a, \hat{x}_t) \leq 0, \quad i = 1, \dots, m, \quad (6)$$

where $c_i(\cdot)$ measures the degree of violation for a specific feasibility condition. An action is feasible only if all constraints are satisfied. The validation function can therefore be written as

$$V(a, \hat{x}_t) = \begin{cases} 1, & \text{if } c_i(a, \hat{x}_t) \leq 0 \text{ for all } i, \\ 0, & \text{otherwise.} \end{cases} \quad (7)$$

The constraint set captures physical and operational limits of the manufacturing system and maps directly to the validation components in Fig. 1. Typical constraint categories include:

- **Kinematic and capability constraints**, which ensure that commanded motions or operating conditions are reachable and remain within actuator, speed, or workspace limits.
- **Interference and safety constraints**, which ensure that predicted system evolution avoids collisions, unsafe proximity, or other forbidden states.
- **Timing and resource constraints**, which ensure that the action can be executed within latency budgets and does not violate scheduling, control, or resource availability requirements.

The emulation twin evaluates these constraints using a *forward emulation model*. Given the current state estimate $\hat{x}_t$ and a candidate action a , this model predicts the resulting system trajectory over a short horizon T :

$$\tilde{x}_{t:t+T} = \mathcal{S}(\hat{x}_t, a), \quad (8)$$

Ibrahim:
 “forward
 emu-
 lation
 model”
 distin-
 guishes
 this
 from
 a full-
 fidelity
 simu-
 lator.
 If you
 prefer a
 different
 term,
 update
 consis-
 tently
 here, in
 Section 3.6,
 and
 in the
 abstract.

where $\mathcal{S}(\cdot)$ denotes the “forward emulation model”. Constraint functions are evaluated on the predicted trajectory rather than the instantaneous state. This allows the validation gate to detect future collisions, infeasible motions, or timing violations before execution.

Based on the validation outcome, the gate implements the approve/reject/modify decision logic illustrated in Fig. 1:

- **Approve:** If $V(a, \hat{x}_t) = 1$, the gate forwards the action to the execution layer.
- **Reject:** If no candidate satisfies all constraints, the gate rejects the candidate set and issues a safe fallback action.
- **Modify:** If a feasible adjustment exists, the gate either returns a repaired candidate that satisfies constraints or requests that the cognitive twin regenerate candidates under updated constraints (Section 3.2).

Uncertainty-aware validation

The deterministic formulation above assumes that $\hat{x}_t$ is known exactly. In many manufacturing settings, future system behavior is uncertain due to sensing noise, model error, or external disturbances. Let ξ_t represent this uncertainty. The constraint functions then depend on ξ_t , and feasibility requirements take probabilistic form:

$$\Pr(c_i(a, \hat{x}_t, \xi_t) \leq 0) \geq 1 - \delta, \quad (9)$$

where δ is a tolerable risk level. Chance-constrained formulations of this type are surveyed by Mesbah (2016). Our emulation twin approximates them through scenario-based simulation consistent with the online validation approach of Negri et al. (2021).

Rather than computing these probabilities exactly, the emulation twin approximates this requirement using conservative predictive models. In practice, the simulator propagates multiple disturbance scenarios or evaluates constraint functions under

bounded uncertainty. Validation is then performed against worst-case or high-risk outcomes, enabling robust feasibility screening while maintaining real-time performance.

In our demonstration instantiation (Section 3.6), uncertainty arises from perception error and short-horizon prediction of dynamic agents. The validation gate accounts for this by evaluating candidate actions against conservative safety envelopes over the prediction horizon.

3.4 Execution and Physical System

The execution layer in Fig. 1 receives validated actions from the validation gate and applies them to the physical system. It contains three components. The control logic implementation translates the approved action a_t into actuator-level commands (e.g., PLC programs, robot motion plans, or process setpoints). The manufacturing system is the physical plant that executes these commands and evolves according to Eq. (5). The system log records executed actions, observed outcomes, and validation decisions to support audit, traceability, and offline analysis. After execution, new sensor observations feed back to the cognitive twin, closing the loop defined in Section 3.1.

3.5 Interfaces and System Considerations

This section should demonstrate that the framework is deployable in a real manufacturing setting. Target length: ~1.5–2.25 pages (without the figure). Do NOT repeat material from Sections 3.1–3.4. Focus on practical system integration issues. Keep the section domain-neutral; Section 3.6 is where the HRC instantiation belongs.

Ibrahim,
please
work
on this
section.

3.5.1 Data Interfaces

Describe the information flows between components. Keep notation consistent with Section 3.1. Cover the following flows: (1) Execution layer $\rightarrow$ Cognitive twin: sensor measurements y_t , system telemetry/status. (2) Cognitive twin $\rightarrow$ Emulation twin: state estimate $\hat{x}_t$, candidate set $\mathcal{A}_t^{\text{cand}}$. (3) Emulation twin $\rightarrow$ Execution layer:

approved action a_t , modified action (if applicable), rejection/fallback command.

(4) Execution layer $\rightarrow$ Logging/monitoring: timestamps, action status, system state for audit or analysis. Also mention: messages should be time-stamped, components may operate at different update rates, refer to data-flow arrows in Fig. 1.

3.5.2 Timing and Real-Time Operation

This part is important since it is related to practical implementation. Explain the timing logic without giving specific numbers yet. Discuss: (1) Decision cycle: sensing $\rightarrow$ estimation $\rightarrow$ candidate generation $\rightarrow$ validation $\rightarrow$ execution. (2) Total latency must be shorter than the system response time. (3) Validation must complete within a fixed time budget. (4) Components may run asynchronously (e.g., sensing faster than validation). (5) If validation exceeds the time budget $\rightarrow$ safe fallback action. Emphasize: the emulation twin is designed to be a real-time feasibility filter, not a computational bottleneck.

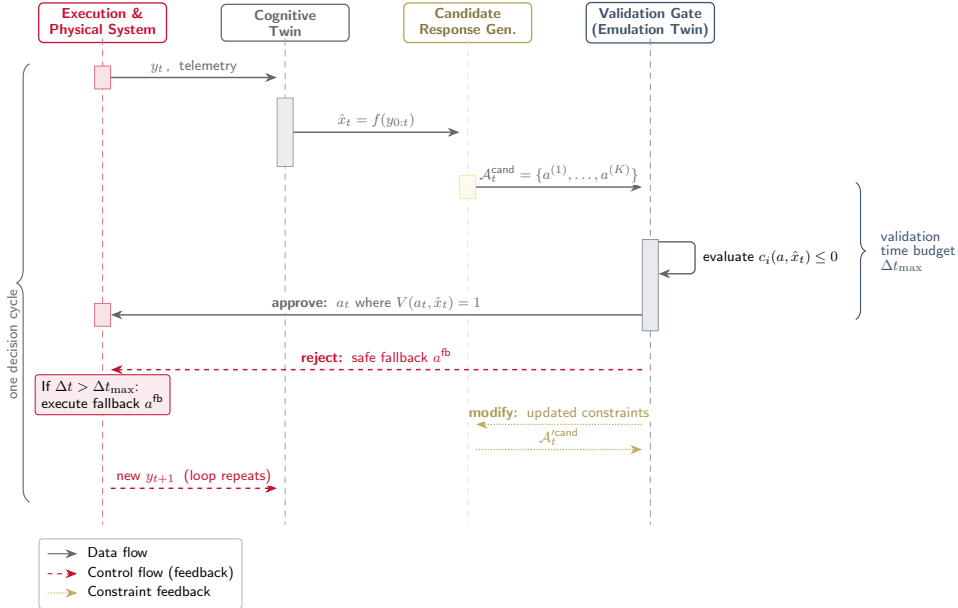


Fig. 2 Decision cycle sequence diagram showing the temporal ordering of messages between architecture components. A full cycle runs from sensing through validation to execution. The validation time budget Δt_{max} bounds the screening latency; a safe fallback action is issued if the budget is exceeded.

3.5.3 Deployment Architecture

Keep this brief (1 paragraph or short bullets). Describe typical compute placement: (1) Time-critical components (estimation + validation) at the edge or local controller. (2) Predictive simulation running on local compute to minimize latency. (3) Heavy tasks (model training, large-scale simulation, analytics) performed offline or in the cloud. Goal: show that the architecture fits modern industrial edge-cloud systems.

3.5.4 Architectural Design Principles

End the section with short bullets (4–5 items) summarizing the engineering philosophy: (1) Separation of inference (cognitive) and feasibility enforcement (emulation). (2) Finite candidate set to enable predictable validation time. (3) Conservative validation under uncertainty. (4) Fail-safe behavior if no candidate is feasible. (5) Modular design so perception/decision methods can be replaced without changing validation.

3.6 HRC Instantiation of the Architecture

We instantiate the cognitive-emulation architecture in a human-robot collaborative assembly task. The implementation uses a UR5e collaborative robot and a single human operator working at a shared table within the *Advanced Manufacturing Workforce and Innovation Hub* at *Miami University*. An Intel *RealSense L515 LiDAR* camera is mounted above the workspace in a top-down configuration to provide a full overhead view of the assembly area. In this task, the human prepares and places assembly parts on a motor base, and the robot then manipulates and assembles those parts. This shared workspace, combined with the sequential handoff between human and robot, creates the safety constraints that the architecture must monitor and enforce.

Human-robot collaboration under speed-and-separation monitoring ([International Organization for Standardization, 2016, 2025](#); [Robotic Industries Association, 2016](#)) places demanding requirements on all layers of the architecture. Human motion is

dynamic and uncertain. The robot’s braking behavior and kinematic limits are physically bounded. Any candidate intervention, such as speed reduction, controlled stop, or workspace reallocation, must therefore be validated against these physical limits within a strict timing budget. This setting exercises all four architectural blocks in Fig. 1 and maps naturally to the constraint categories introduced in Section 3.3.

State estimation

The cognitive twin implements $\hat{x}_t = f(y_{0:t})$ from Eq. (3) through vision-based perception. The overhead *RealSense L515* streams RGB and depth frames at 30 fps to a *YOLOv11* instance-segmentation model. The model was trained on 2,000 labeled images for 200 epochs using an *Nvidia RTX 5000 GPU* and detects four classes: human, robot, motor base, and individual assembly parts. It achieved a bounding-box mean average precision (mAP) at 0.50 intersection-over-union (IoU) of 0.995 and a mask mAP@50–95 of 0.909. Using these detections, the system constructs the state estimate $\hat{x}_t$, including the estimated human hand position, the robot joint configuration read from the controller, and the locations of parts placed on the motor base. We selected instance segmentation rather than bounding-box detection because pixel-level masks are needed to define fine-grained zone boundaries.

Candidate generation

Based on $\hat{x}_t$, the cognitive twin generates a finite set of candidate responses $\mathcal{A}_t^{\text{cand}} = \pi(\hat{x}_t)$ from Eq. (4). These candidates include discrete speed-reduction levels, such as 75%, 50%, 25%, and 10% of nominal robot speed, a safety-rated controlled stop, a dynamic workspace-boundary adjustment, and an escalation signal requesting human operator review. We use discrete alternatives instead of continuous optimization so that the validation gate can evaluate each candidate within a predictable time budget.

Constraint functions

The emulation twin evaluates candidate actions using four constraint functions. Note that each of those constraints are aligned with one of the categories introduced in Section 3.3.

The first constraint encodes the *protective separation distance* prescribed by [Robotic Industries Association \(2016\)](#), which defined the minimum safe separation as

$$S_p(t_0) = S_h + S_r + S_s + C + Z_d + Z_r, \quad (10)$$

where S_h accounts for the operator’s motion during the robot’s reaction and stopping interval, S_r and S_s represent robot motion during the reaction and braking phases, C is the intrusion distance before detection, and Z_d and Z_r capture uncertainty in the sensing system and robot position, respectively. Given a candidate action a and the current state estimate $\hat{x}_t$, the separation-distance constraint is written as

$$c_{\text{sep}}(a, \hat{x}_t) = S_p(t_0) - d(a, \hat{x}_t) \leq 0, \quad (11)$$

where $d(a, \hat{x}_t)$ is the predicted minimum distance between any hazardous robot part and the operator under action a . The cognitive twin supplies the human speed estimate v_h needed for S_h , the forward emulation model $\mathcal{S}(\cdot)$ provides the robot speed and braking trajectory needed for S_r and S_s . C , Z_d and Z_r are calibrated parameters.

The second constraint is a *zone-violation check*. The motor base is partitioned into a robot zone, a human zone, and an intermediate buffer zone. These boundaries are defined by arcs centered on the robot base so that they reflect the manipulator’s reach envelope rather than arbitrary linear partitions. The buffer zone absorbs minor perception noise and provides an added safety margin beyond the static terms $C +$

$Z_d + Z_r$ in Eq. (10). A zone-violation constraint

$$c_{\text{zone}}(a, \hat{x}_t) \leq 0 \quad (12)$$

is satisfied when the predicted robot trajectory under action a remains inside the robot zone and the human remains outside it. This zone-based formulation can be viewed as a spatial discretization of the continuous separation-distance requirement, since it enforces $d(a, \hat{x}_t) \geq S_p(t_0)$, with the buffer absorbing the dynamic margin.

The third and fourth constraints address implementation feasibility by ensuring that each candidate intervention is both physically executable and fast enough to support real-time operation. The third constraint is a *kinematic feasibility check*, $c_{\text{kin}}(a, \hat{x}_t) \leq 0$, which verifies that a candidate speed or stop command can be executed within the UR5e’s joint limits, velocity bounds, and workspace geometry. The fourth constraint is a *latency bound*, $c_{\text{lat}}(a, \hat{x}_t) \leq 0$, which ensures that the elapsed time from candidate generation through validation remains within the budget Δt_{max} shown in Fig. 2.

Validation gate

The validation gate applies $V(a, \hat{x}_t)$ from Eq. (7) to each candidate. If all four constraints are satisfied, the gate approves the action and forwards it to the execution layer. If no candidate passes, the gate issues a safe fallback action, namely a controlled stop. If a candidate is close to feasible but fails on a limited subset of constraints, for example because the requested speed is too high, the gate attempts a modification by selecting a lower speed level and re-evaluating. If that revised action also fails, the gate asks the candidate-response generator to produce a new set under the tightened constraints, as described in Section 3.2.

Dynamic zone reallocation

Zone boundaries are allowed to change over time. When the human operator completes a subtask and places one or more parts on the motor base, the robot may need access to that newly occupied region. The cognitive twin detects placed parts from the YOLOv11 segmentation output. In particular, a part whose centroid falls within the motor base’s oriented bounding box triggers a transition. The system then recomputes the zone boundaries by extending the robot-zone arc to include the furthest placed part, resizing the buffer when space permits, and assigning the remaining area to the human zone. This reallocation is itself validated. Before the robot is allowed to operate in the expanded region, the emulation twin checks c_{zone} and c_{sep} against the proposed boundaries. If the human is still present in the area to be absorbed, the reallocation is deferred until c_{sep} is satisfied.

Risk assessment as discretized constraint screening

An earlier version of the system used a likelihood $\times$ severity risk-assessment matrix to score detected safety events and select interventions. The constraint-based formulation presented here generalizes that approach. The matrix provided a coarse discretization of quantities that the constraint functions $c_i(a, \hat{x}_t)$ now evaluate continuously. In this mapping, likelihood corresponds to predicted proximity from $\mathcal{S}(\hat{x}_t, a)$, while severity corresponds to the magnitude of constraint violation. The binary validation function V replaces the matrix lookup with an explicit and auditable feasibility check. We retain the matrix as a fallback decision policy in the event of sensor or model failure, but the primary decision path uses constraint screening.

Forward emulation model

The forward emulation model $\mathcal{S}(\hat{x}_t, a)$ from Eq. (8) is implemented in NVIDIA Isaac Sim. We selected this environment using criteria aligned with the systematic evaluation of digital twin platforms reported by [Doderio et al. \(2025\)](#), who compared sixteen

development environments across robotics, game, and simulation engines. Isaac Sim provides a production-grade kinematic and collision model for the UR5e, supports short-horizon trajectory prediction over the interval T , and integrates with ROS2 for real-time data exchange. The implementation reproduces the UR5e geometry, joint limits, and velocity constraints. An overhead virtual camera approximates the RealSense sensor’s field of view for depth-map validation. Given a candidate action a , Isaac Sim predicts the resulting joint trajectory $\tilde{x}_{t:t+T}$ over the next several hundred milliseconds. The constraint functions c_{sep} , c_{zone} , c_{kin} , and c_{lat} are then evaluated on that predicted trajectory before any command reaches the physical robot.

4 Experimental Evaluation

This section needs to show that the cognitive-emulation architecture works as a system, not just as a collection of individual components. The main goal is to demonstrate that the perception pipeline provides state estimates that are accurate and fast enough for downstream use, that the validation gate screens candidate actions correctly against the four constraint functions, and that the full pipeline satisfies real-time requirements while preserving safety margins. I would aim for roughly 4–6 pages here, including figures and tables.

4.1 Experimental Setup

Start with the protocol rather than the hardware. The workcell itself is already introduced in Section 3.6, so there is no need to repeat the equipment description unless a detail is directly relevant to measurement or experimental control. What the reader needs here is a clear account of how the experiments were run.

I would begin by describing the task scenario or scenarios in concrete terms: what assembly sequence the operator performs, how many trials were conducted, and whether the revised study involved one operator or multiple operators. It would also

help to specify the controlled variables up front, including the nominal robot speed profile, camera frame rate, and the Isaac Sim prediction horizon T .

A clean way to organize the experiments is around three scenario categories. The first is **normal operation**, where the operator remains in the human zone and the robot remains in the robot zone without any proximity violation. The second is **close-proximity events**, where the operator intentionally reaches into or near the buffer or robot zone so that c_{sep} and c_{zone} are expected to trigger speed reduction or a controlled stop. The third is **dynamic zone reallocation**, where the operator places parts on the table, the zone boundary is updated, and the robot must wait until c_{sep} is satisfied before entering the expanded zone.

Ground truth also needs to be explained carefully. Please specify how human position, separation distance, and zone occupancy are established for evaluation. This could be done through manual annotation from video or ruler-based calibration markers on the table. Also include the number of trials and trial duration for each scenario category. As a practical minimum, I would suggest at least 10 trials per category, with each trial lasting roughly 30–60 seconds so that there are enough events for meaningful reporting.

4.2 Perception Evaluation

The point of this subsection is not to revisit object-detection performance in the abstract, but to show whether perception is good enough for the downstream safety logic. Since the mAP results are already reported in Section 3.6, the emphasis here should shift to task-relevant perception quality.

A good starting point is **human localization error**. Compare the estimated hand or body centroid derived from the YOLOv11 masks against ground truth, and report both mean error and 95th-percentile error in centimeters. This is important because

localization error directly affects the estimate of S_h and the separation term $d(a, \hat{x}_t)$ in Eq. (11).

The second item I would include is **part detection reliability** for the zone-reallocation experiments. What matters here is whether the system correctly determines that a part has been placed. Precision and recall for the “part placed” event would make sense.

The third metric should be **inference latency**, measured as per-frame inference time on the RTX 5000, since that latency contributes directly to c_{lat} and therefore affects whether the overall pipeline can run in real time.

This subsection should likely include one summary table and a small number of representative qualitative examples. Annotated frames with masks, detected parts, and zone overlays would work well too in my opinion.

4.3 Constraint Evaluation

This is the place to show whether the four constraint functions behave the way they are intended to behave. The main question is straightforward: does each constraint activate when it should, and remain inactive when it should?

For **separation distance**, use the close-proximity trials to plot the predicted separation distance $d(a, \hat{x}_t)$ against the required threshold $S_p(t_0)$ over time. The reader should be able to see clearly when the constraint transitions from satisfied to violated as the operator approaches the robot. At the same time, show that the corresponding intervention, such as speed reduction or stop, is triggered appropriately. Please report true positives, false positives, and false negatives for separation-constraint violation detection. If feasible, also compare predicted distance against a ground-truth measured distance (using a metric like MAE for example and/or showing a histogram of the distribution).

For **zone violation**, use the normal-operation and close-proximity trials to log zone occupancy over time. The goal is to show that zone violations are detected when they occur and that the system responds appropriately. Per-frame zone classification accuracy against ground truth would be useful here.

For **kinematic feasibility**, the emphasis should be on whether approved commands are truly executable on the UR5e. One direct way to validate this is to compare commanded versus actual joint velocities and confirm that approved speed-reduction and stop commands stay within the robot’s joint and velocity limits. The most important result to state explicitly is whether any kinematically infeasible commands were ever forwarded to the robot.

For **latency**, measure end-to-end pipeline latency from frame capture to action dispatch. I would strongly recommend breaking this down into at least four parts: inference time, candidate generation time, Isaac Sim validation time, and communication overhead. Report mean, standard deviation, and worst-case latency, and check whether the worst case remains below $\Delta t_{\max}$.

4.4 Validation Gate Evaluation

This is the central experiment in the section because it tests the core claim of our paper. What you want to establish here is that the validation gate improves decision quality relative to sending candidate actions directly to execution.

I would begin with the **decision outcome distribution**. Across all trials, report the fraction of candidate sets that were approved as-is, modified by the gate, or rejected in favor of the fallback action. This should be broken down by scenario category so that the reader can see how the gate behaves under different conditions.

The most important comparison is **with versus without the validation gate**. Run the same scenarios with the validation gate disabled, meaning that the cognitive twin’s top-ranked candidate is passed directly to execution. Then compare the two

settings along three dimensions. First, compare **safety margin**, using the minimum observed separation distance. With the gate active, this should not fall below $S_p(t_0)$; without the gate, violations may occur. Second, compare the number of **infeasible commands**, meaning actions that violate kinematic or latency constraints but still reach the robot. Third, compare **productivity impact**, such as robot idle time or total task completion time, so that the paper can address whether the gate introduces unnecessary conservatism during normal operation.

If time allows, I would also recommend a comparison between **constraint screening and the earlier risk-matrix policy** (i.e., the one in the original NAMRC submission). Since Section 3.6 mentions the risk matrix as the predecessor, this would help show why the new decision logic is preferable. In particular, it would be useful to show that the constraint-based approach gives more granular and defensible responses, such as graded speed reduction rather than simple stop/go behavior. A summary table comparing these conditions would be a good way to present the results.

4.5 Dynamic Zone Reallocation

This part should treat zone reallocation as a distinct capability of the system rather than as a side detail. The main thing to show is that the transition is detected correctly, validated correctly, and deferred when safety requires it.

Start with **transition correctness**. For each part-placement event, log whether the system correctly detected the event, recomputed the zone boundary, validated the proposed new boundary, and only then allowed the robot to proceed.

The next quantity to report is **transition latency**, defined as the time from part detection to robot entry into the expanded zone. That latency should include validation time and any waiting period caused by the human remaining near the region to be absorbed.

It is also important to show that safety is maintained throughout the transition itself. In particular, the experiments should show that c_{sep} remains enforced during the full reallocation process. If the operator is still near the newly allocated region, the transition should be deferred rather than executed immediately.

A timeline figure would be especially effective here. For one representative trial, show how the zone boundary changes over time, together with human position and robot state.

4.6 Results Summary

Use the final subsection to synthesize the main findings rather than restate all of the detailed results. A compact summary table would work well, with rows corresponding to metrics and columns corresponding to scenario categories or experimental conditions.

The accompanying text should highlight only the main takeaways: that perception is accurate and fast enough for downstream use, that all four constraints behave as intended, that end-to-end latency remains within Δt_{max} , that the validation gate prevents actions that would otherwise violate safety or feasibility requirements, and that dynamic zone reallocation works correctly, including deferred transitions when the human is still present. This part should stay brief. The detailed evidence should already appear in Sections 4.2–4.5.

Supplementary information. If your article has accompanying supplementary file/s please state so here.

Authors reporting data from electrophoretic gels and blots should supply the full unprocessed scans for key as part of their Supplementary information. This may be requested by the editorial team/s if it is missing.

Please refer to Journal-level guidance for any specific requirements.

Acknowledgements. Acknowledgements are not compulsory. Where included they should be brief. Grant or contribution numbers may be acknowledged.

Please refer to Journal-level guidance for any specific requirements.

Appendix A Section title of first appendix

An appendix contains supplementary information that is not an essential part of the text itself but which may be helpful in providing a more comprehensive understanding of the research problem or it is information that is too cumbersome to be included in the body of the paper.

References

- Alenjareghi, M.J., Keivanpour, S., Chinniah, Y.A., Jocelyn, S. (2025). Computer vision-enabled real-time job hazard analysis for safe human–robot collaboration in disassembly tasks. *Journal of Intelligent Manufacturing*, 36(8), 5563–5591, <https://doi.org/10.1007/s10845-024-02519-8>
- Chand, S., Zheng, H., Lu, Y. (2024). A vision-enabled fatigue-sensitive human digital twin towards human-centric human–robot collaboration. *Journal of Manufacturing Systems*, 77, 432–445, <https://doi.org/10.1016/j.jmsy.2024.10.002>
- Chebotareva, E., Mustafin, M., Safin, R., Tsoy, T., Martinez-García, E.A., Li, H., Magid, E. (2025). Camera-based safety system for collaborative assembly. *Journal of Intelligent Manufacturing*, 36(8), 5593–5611, <https://doi.org/10.1007/s10845-024-02533-w>

- Cimino, A., Longo, F., Nicoletti, L., Solina, V. (2024). Simulation-based digital twin for enhancing human–robot collaboration in assembly systems. *Journal of Manufacturing Systems*, 77, 1027–1044, <https://doi.org/10.1016/j.jmsy.2024.10.024>
- Dodero, C., McCormick, M.R., Malik, A.A., Harik, R., Wuest, T. (2025). Systematic selection framework for digital twin development environments in smart manufacturing. *Digital Twin*, 2(4), 2565246, <https://doi.org/10.1080/27525783.2025.2565246>
- El Kalach, F., Farahani, M.A., Samaha, P., Wuest, T., Harik, R. (2026). Digital twin enabled robot collision detection using time series forecasting. *Journal of Intelligent Manufacturing*, 1–18, <https://doi.org/10.1007/s10845-026-02803-9> (Ahead of Print)
- Faqeer, H.A., & Khajavi, S.H. (2025). Digital twin and computer vision combination for manufacturing and operations: A systematic literature review. *Applied Sciences*, 15(18), 10157, <https://doi.org/10.3390/app151810157>
- Gálvez del Postigo Gallego, L., Lazaro-Molnar, S., del Real Torres, A., Acevedo Galicia, L. (2025). Decision support within digital twins in manufacturing ecosystems: A review. *IET Collaborative Intelligent Manufacturing*, 7(e70041), 1–25, <https://doi.org/10.1049/cim2.70041>
- Grieves, M., & Vickers, J. (2017). Digital twin: Mitigating unpredictable, undesirable emergent behavior in complex systems. In F.-J. Kahlen, S. Flumerfelt, & A. Alves (Eds.), *Transdisciplinary perspectives on complex systems: New findings and approaches* (pp. 85–113). Cham: Springer International Publishing.
- International Organization for Standardization (2016). *ISO/TS 15066:2016 robots and robotic devices — collaborative robots*. International Organization for Standardization (ISO). Retrieved from <https://www.iso.org/standard/62996.html>. (Standard. Accessed: February 28, 2026)
- International Organization for Standardization (2021). *ISO 23247-1:2021 automation systems and integration — digital twin framework for manufacturing — part 1: Overview and general principles*. International Organization for Standardization (ISO). Retrieved from <https://www.iso.org/standard/75066.html>. (Standard. Accessed: February 28, 2026)
- International Organization for Standardization (2025). *ISO 10218-1:2025 robotics – safety requirements – part 1: Industrial robots*. International Organization

- for Standardization (ISO). Retrieved from <https://www.iso.org/standard/73933.html>. (Standard. Accessed: March 15, 2026)
- Iranshahi, K., Brun, J., Arnold, T., Sergi, T., Müller, U.C. (2025). Digital twins: Recent advances and future directions in engineering fields. *Intelligent Systems with Applications*, 26, 200516, <https://doi.org/10.1016/j.iswa.2025.200516>
- Jones, D., Snider, C., Nassehi, A., Yon, J., Hicks, B. (2020). Characterising the digital twin: A systematic literature review. *CIRP Journal of Manufacturing Science and Technology*, 29(A), 36–52, <https://doi.org/10.1016/j.cirpj.2020.02.002>
- Keshvarparast, A., Battini, D., Battaia, O., Pirayesh, A., Dolgui, A. (2024). Collaborative robots in manufacturing and assembly systems: literature review and future research agenda. *Journal of Intelligent Manufacturing*, 35(5), 2065–2118, <https://doi.org/10.1007/s10845-023-02137-w>
- Kritzing, W., Karner, M., Traar, G., Henjes, J., Sihn, W. (2018). Digital twin in manufacturing: A categorical literature review and classification. *IFAC-PapersOnLine*, 51(11), 1016–1022, <https://doi.org/10.1016/j.ifacol.2018.08.474>
- Langâs, E.F., Zafar, M.H., Sanfilippo, F. (2026). Exploring the synergy of human-robot teaming, digital twins, and machine learning in industry 5.0: a step towards sustainable manufacturing. *Journal of Intelligent Manufacturing*, 37, 999–1022, <https://doi.org/10.1007/s10845-025-02580-x>
- Li, W., Hu, Y., Zhou, Y., Pham, D.T. (2024). Safe human–robot collaboration for industrial settings: a survey. *Journal of Intelligent Manufacturing*, 35, 2235–2261, <https://doi.org/10.1007/s10845-023-02159-4>
- Li, Z., Mei, X., Sun, Z., et al. (2025). A reference framework for the digital twin smart factory based on cloud-fog-edge computing collaboration. *Journal of Intelligent Manufacturing*, 36, 3625–3645, <https://doi.org/10.1007/s10845-024-02424-0>
- Lugaresi, G., Gangemi, S., Gazzoni, G., Matta, A. (2023). Online validation of digital twins for manufacturing systems. *Computers in Industry*, 150, 103942, <https://doi.org/10.1016/j.compind.2023.103942>

- Macháček, Z., Hercik, R., Václavík, A., Zemanek, J., Hameed, I.A., Koziorek, J. (2025). Modern trends and industrial use cases of digital twin technology with 3D behavioral representation. *Journal of Intelligent Manufacturing*, , <https://doi.org/10.1007/s10845-025-02709-y> (Ahead of print.)
- Marvel, J.A., & Norcross, R. (2017). Implementing speed and separation monitoring in collaborative robot workcells. *Robotics and Computer-Integrated Manufacturing*, 44, 144–155, <https://doi.org/10.1016/j.rcim.2016.08.001>
- Mayne, D.Q., Rawlings, J.B., Rao, C.V., Sokaert, P.O. (2000). Constrained model predictive control: Stability and optimality. *Automatica*, 36(6), 789–814, [https://doi.org/10.1016/S0005-1098\(99\)00214-9](https://doi.org/10.1016/S0005-1098(99)00214-9)
- Mesbah, A. (2016). Stochastic model predictive control: An overview and perspectives for future research. *IEEE Control Systems Magazine*, 36(6), 30–44, <https://doi.org/10.1109/MCS.2016.2602087>
- Negri, E., Pandhare, V., Cattaneo, L., Singh, J., Macchi, M., Lee, J. (2021). Field-synchronized digital twin framework for production scheduling with uncertainty. *Journal of Intelligent Manufacturing*, 32(4), 1207–1228, <https://doi.org/10.1007/s10845-020-01685-9>
- NVIDIA (2026a). *Assembling digital twins with Omniverse and OpenUSD*. Documentation / learning path. Retrieved from <https://docs.omniverse.nvidia.com/digital-twins/latest/index.html>. (Accessed: February 28, 2026)
- NVIDIA (2026b). *Digital twin — Isaac Sim documentation (v6.0.0)*. Documentation. Retrieved from https://docs.isaacsim.omniverse.nvidia.com/6.0.0/digital_twin/index.html. (Accessed: February 28, 2026)
- Pronost, G., Mayer, F., Camargo, M., Dupont, L. (2024). Digital twins along the product lifecycle: A systematic literature review of applications in manufacturing. *Digital Twin*, 3(3), , <https://doi.org/10.12688/digitaltwin.17807.2>
- Ramkumar, A., & Balasubramanian, G. (2025). Deep convolutional neural network object net model based cognitive digital twin for trust in human–robot collaborative manufacturing. *Journal of Intelligent Manufacturing*, 36, 5141–5161, <https://doi.org/10.1007/s10845-024-02501-4>

- Redelinghuys, A.J.H., Basson, A.H., Kruger, K. (2020). A six-layer architecture for the digital twin: a manufacturing case study implementation. *Journal of Intelligent Manufacturing*, 31, 1383–1402, <https://doi.org/10.1007/s10845-019-01516-6>
- Robotic Industries Association (2016). *RIA TR R15.606-2016 technical report for industrial robots and robot systems – safety requirements – collaborative robots*. Robotic Industries Association, Ann Arbor, MI. (Registered with ANSI, December 25, 2016)
- Saleem, Z., Gustafsson, F., Furey, E., McAfee, M., Huq, S. (2025). A review of external sensors for human detection in a human robot collaborative environment. *Journal of Intelligent Manufacturing*, 36, 2255–2279, <https://doi.org/10.1007/s10845-024-02341-2>
- Seipolt, A., Buschermöhle, R., Hasselbring, W. (2025). Digital twins in manufacturing: A systematic literature review with retrieval-augmented generation. *IEEE Access*, 13, 172562–172583, <https://doi.org/10.1109/ACCESS.2025.3611269>
- Shao, G. (2024). *Manufacturing digital twin standards*. NIST publication page. Retrieved from <https://www.nist.gov/publications/manufacturing-digital-twin-standards>. (Accessed: February 28, 2026)
- Siemens (2026a). *Process simulate x robotics standard*. Product documentation page. Retrieved from <https://www.siemens.com/en-us/products/tecnomatix/offerings/process-simulate-x-robotics-standard/>. (Accessed: February 28, 2026)
- Siemens (2026b). *Process simulate x safety robots manager*. Product documentation page. Retrieved from <https://www.siemens.com/en-us/products/tecnomatix/offerings/process-simulate-x-safety-robots-manager/>. (Accessed: February 28, 2026)
- Szabo, S., Shackelford, W., Norcross, R., Marvel, J.A. (2012). *A testbed for evaluation of speed and separation monitoring in a human robot collaborative environment*. NIST IR 7851, National Institute of Standards and Technology (NIST). Retrieved from <https://doi.org/10.6028/NIST.IR.7851>. (Accessed: February 28, 2026)
- Tao, F., Cheng, J., Qi, Q., Zhang, M., Zhang, H., Sui, F. (2018). Digital twin-driven product design, manufacturing and service with big data. *The International Journal of Advanced Manufacturing Technology*, 94, 3563–3576, <https://doi.org/10.1007/s00170-017-0233-1>

- Tong, X., Liu, Q., Pi, S., Xiao, Y. (2020). Real-time machining data application and service based on IMT digital twin. *Journal of Intelligent Manufacturing*, 31, 1113–1132, <https://doi.org/10.1007/s10845-019-01500-0>
- Voas, J., et al. (2025). *NIST IR 8356: Security and trust considerations for digital twin technology*. National Institute of Standards and Technology (NIST). Retrieved from <https://nvlpubs.nist.gov/nistpubs/ir/2025/NIST.IR.8356.pdf>. (Accessed: February 28, 2026)
- Wang, H., Lv, L., Li, X., Li, H., Leng, J., Zhang, Y., ... Hu, G. (2023). A safety management approach for Industry 5.0's human-centered manufacturing based on digital twin. *Journal of Manufacturing Systems*, 66, 1–12, <https://doi.org/10.1016/j.jmsy.2022.11.013>